In-Class Activity 2: Using SRILM (Brown Corpus) to Estimate Word Probabilities

Roll No.: G25AIT2067 Course: Foundations of Vision and Language Code: G25AIT2067-colab.ipynb

Setup: steps 1 to 3 (count, train, test)

Brown corpus: 37,272 training, 5,734 dev and 14,334 test sentences; 40,604 training types; the 305,056-token test set has 18,639 OOV words (6.11%).

The next three parts of the tutorial were carried out:

Step 1 ./ngram-count -text browndata/brown-train.txt -order 2 -write brown-train.count

Step 2 ./ngram-count -read brown-train.count -order 2 -lm brown-train.lm -addsmooth 0 -unk

Step 3 ./ngram -lm brown-train.lm -ppl browndata/brown-test.txt -debug 2

We checked the count file before using it: tr | grep -cx the finds 38,810 occurrences of "the" in the raw text, matching the count file, and the 37,272 non-empty training lines equal the <s> markers in the count file. The plain bigram on the test set gave 18,639 OOVs, 39,856 zeroprobs, logprob −990,704, ppl 6270.9 (ppl1 10425.3). The following questions are based on this run.

Q1: What are the meanings of -inf for OOV words, and of -unk and -addsmooth 0?

The problem is -addsmooth 0. It turns smoothing off, so a word not in the training set gets probability 0, as does a bigram whose history has no spare mass, and log 0 is -inf. SRILM doesn't stop there: it drops those words, counts them as OOVs and zeroprobs, and computes perplexity on the rest, so 6270.9 looks better than it should. We hoped that -unk would help, but it didn't; it reserves an <unk> slot, but with smoothing off that slot also has count 0 (log P = −99). They're used for different purposes: one for assigning a probability to unseen events, one for labelling unknown words. With -unk plus actual smoothing the zeros are gone: with add-1 we had 0 OOVs, 0 zeroprobs and a ppl of 2493.2 for the whole test set.

Q2: Other smoothing methods: perplexity and OOV probability

Train with:

./ngram-count -text browndata/brown-train.txt -order 2 <smoothing> -unk -lm m.lm

then test with:

./ngram -lm m.lm -ppl browndata/brown-test.txt

Each method was run twice, open (-unk) and closed (no -unk).

Smoothing method	P(<unk>)	Bigram open ppl	Bigram closed ppl
Add-1 (-addsmooth 1)	0.000004	2493.2	1881.1
Add-0.01 (-addsmooth 0.01)	0.000004	877.7	553.3
Good-Turing (default)	0.000004	642.3	422.2
Witten-Bell (-wbdiscount)	0.050	396.8	438.3
Kneser-Ney (-kndiscount)	0.133	352.5	401.8
Interp. modified KN (-kndiscount -interpolate)	0.000003	617.9	393.5

The closed run has no -unk, so words outside the vocabulary are not counted and all methods are tested on the same in-vocabulary words; the open run has -unk on. The reliable column is the closed one, where interpolated modified Kneser-Ney ranks highest at 393.5 and add-1 lowest at 1881.1, as it allocates too much probability to bigrams that do not occur. In the open column it looks like Witten-Bell and plain Kneser-Ney are ahead, but this is deceptive. Both are back-off models that place a high percentage of the unigram mass, 5% and 13% respectively, in <unk>, so the unknown test tokens are cheap and the perplexity falls for the wrong reason. The takeaway: only models that share the same vocabulary can be compared on perplexity.

Q3: Trigram (n = 3) instead of bigram

Train with:

./ngram-count -text browndata/brown-train.txt -order 3 <smoothing> -lm m3.lm

then test with:

./ngram -order 3 -lm m3.lm -ppl browndata/brown-test.txt

Closed vocabulary: the same words as in Q2.

Method	Bigram ppl	Trigram ppl
Add-1 (-addsmooth 1)	1881.1	2798.0
Good-Turing (default)	422.2	411.8
Witten-Bell (-wbdiscount)	438.3	423.0
Interp. modified KN (-kndiscount -interpolate)	393.5	374.3

Each sensible smoother benefited from going to a trigram: interpolated KN went from 393.5 to 374.3, Good-Turing from 422.2 to 411.8, Witten-Bell from 438.3 to 423.0, as the additional word of context reduces some ambiguity. The odd one out is add-1, which gets worse, 1881 to 2798: there are many more unseen contexts for trigrams, and add-1 distributes its probability over all of them.

Q4: Removal of punctuation and stop words for pre-processing

Use the same pre-processing for the train and test sets and train using:

./ngram-count -text train.txt -order 2 -kndiscount -interpolate -lm pp.lm

and test with the matching test file (closed vocabulary).

Pre-processing (train and test)	Vocabulary	Test tokens	OOV %	ppl
Original	40,604	305,056	6.11	393.5
Lower-case	36,585	305,056	5.26	383.2
Lower-case + no punctuation	36,568	268,236	5.98	612.5
Lower-case + no punctuation + no stop-words	36,440	183,000	9.29	2990.8

Lowercasing helped: the vocabulary was reduced by about 10%, fewer test words fell outside it, and ppl was reduced to 383.2. Interestingly, when we take the punctuation out, ppl increases to 612.5, because commas and full stops are among the most predictable tokens, and deleting them leaves harder ones. Removing the stop words had the most significant effect, with ppl reaching 2990.8, as these very common words were the most predictable and the model was left to predict content words at 9.3% OOV. These numbers indicate the predictability of the rest of the text, not whether a model improved, since each setting scores different tokens.

Q5: Tuned interpolated model (unigram + bigram) on brown-dev

(a) Train:

./ngram-count -text brown-train.txt -order 1 -addsmooth k -unk -lm uni.lm

./ngram-count -text brown-train.txt -order 2 -addsmooth k -unk -lm bi.lm

(b, c) Tune on the dev set:

./ngram -lm bi.lm -mix-lm uni.lm -lambda L -ppl browndata/brown-dev.txt

for k in {0.001, 0.01, 0.1} and L in {0, 0.1, …, 1}.

(d) Run the best setting again with -ppl browndata/brown-test.txt.

We trained a unigram and a bigram for each small -addsmooth value and combined them with -mix-lm, varying the bigram weight λ from 0 to 1 in steps of 0.1 on brown-dev. The best cell is $k = 0.001$, $\lambda = 0.7$, dev ppl 581.0; the unigram alone gives 1218.9 and the bigram alone 800.5, so the mixture clearly beats either. On the untouched test set the interpolated model scored 681.3, against 963.1 for the bigram and 1362.5 for the unigram. If the bigram is rare or absent, the unigram is used instead. Note: the option is -lambda, not -lamda.